

CI/CD Pipeline Exploitation response

Supply Chain / DevSecOps · **Critical** severity · SOC IR Playbook

Incident type	Software Supply Chain / Pipeline Compromise
Severity	Critical
Priority	Critical
Detection sources	SIEM, Source Control Logs, CI/CD Logs, EDR, SAST/DAST Tools, Developer Reports, Threat Intel Feeds

Scenario

An attacker gains access to or exploits weaknesses in a CI/CD pipeline (e.g., Jenkins, GitLab CI, GitHub Actions) to manipulate build processes, inject malicious code or secrets or use the pipeline to pivot into broader infrastructure.

MITRE ATT&CK

T1059.006, T1136.003, T1556, T1587.002

Preparation

Enforce RBAC on CI/CD tools Restrict who can create or modify pipelines, runners or secrets Enable detailed audit logging For code pushes, pipeline changes, job execution and secret use Use signed commits and artifacts Validate authenticity of source and deployment packages Monitor for pipeline abuse patterns e.g., unexpected job triggers, privilege escalation via runners Isolate build environments Use ephemeral containers/VMs to limit lateral movement and access scope

Detection & Analysis

Alert from SIEM or DevSecOps tools Unexpected job behaviour, credential use or build script changes Review recent pipeline changes Check job definitions, runner configurations and injected commands Analyse source repo activity Look for rogue commits, PRs or branch manipulations Identify affected projects Determine what builds/deployments may have been compromised

Containment

Disable affected pipelines or runners Stop further execution of compromised jobs Revoke access to CI/CD tool For compromised accounts or tokens Block malicious artifacts Prevent deployment of compromised containers, binaries or packages Isolate affected environments Temporarily remove impacted apps or services from the deployment path

Eradication

Clean malicious code or scripts Revert to clean repo state; delete tampered build definitions Rotate compromised secrets Reissue API keys, cloud tokens, database credentials exposed in CI/CD logs Patch vulnerabilities Address misconfigurations in runners, plugins or access control Audit third-party integrations Remove or review access granted to external CI/CD plugins or services

Recovery

Restore trusted pipelines After validating scripts, dependencies and configurations Rebuild affected applications Using known-good code and secured CI/CD process Re-enable deployment Once verified safe and complete validation is passed Notify stakeholders Inform developers, product owners and security teams of the recovery status

Lessons Learned & Reporting

Perform a root cause analysis Determine whether the entry point was repo access, runner abuse or plugin compromise Update pipeline security controls Enforce code review, job approvals and secure secret management

Train DevOps and developers On secure CI/CD practices and incident indicators Report if necessary If data was exposed or software shipped with malware (e.g., to customers, regulators) Update playbooks Include lessons learned and control enhancements for CI/CD monitoring

Tools typically involved

CI/CD platforms (e.g., Jenkins, GitHub Actions, GitLab CI, Azure DevOps); SIEM (e.g., Splunk, Sentinel); EDR and Runtime protection (e.g., CrowdStrike, Aqua Security); Code and pipeline scanners (e.g., SonarQube, Checkov, TFSec); Source code management systems (e.g., GitHub, GitLab, Bitbucket); SOAR platforms (for auto-remediation of pipeline abuse)

Success metrics (SLA targets)

Detection Time <10 minutes from abnormal CI/CD activity Job Disablement Time <30 minutes from confirmation Secret Rotation Time <2 hours from exposure detection Rebuild & Redeploy Time Within 24-48 hours using verified code CI/CD Access Review Completion 100% of user and integration access audited within 3 days